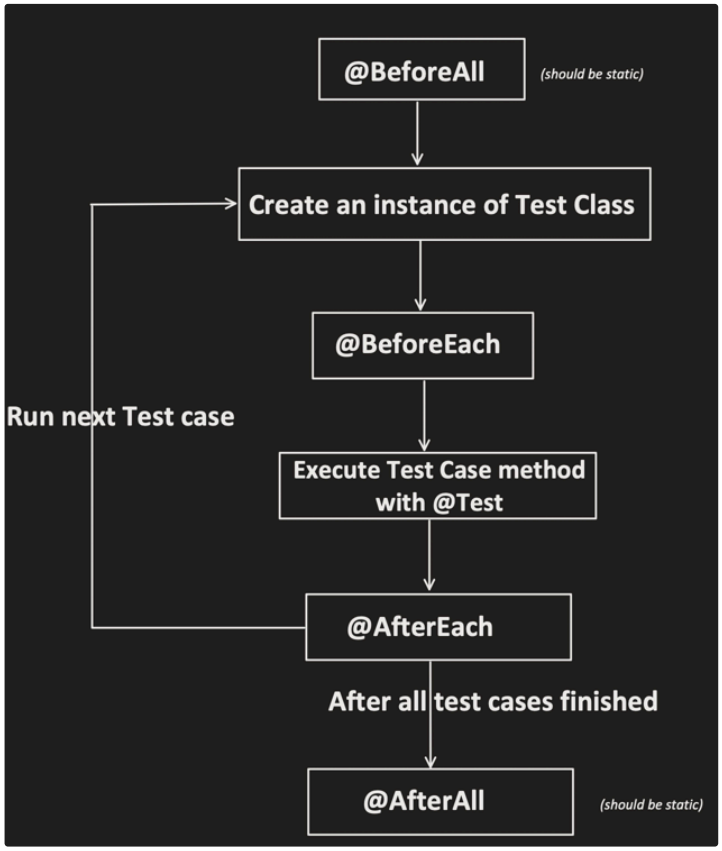
Test Case: Lifecycle, Parallel execution and AAA pattern

Lifecycle of a Junit5 test case:

Lifecycle.PER_METHOD(default)

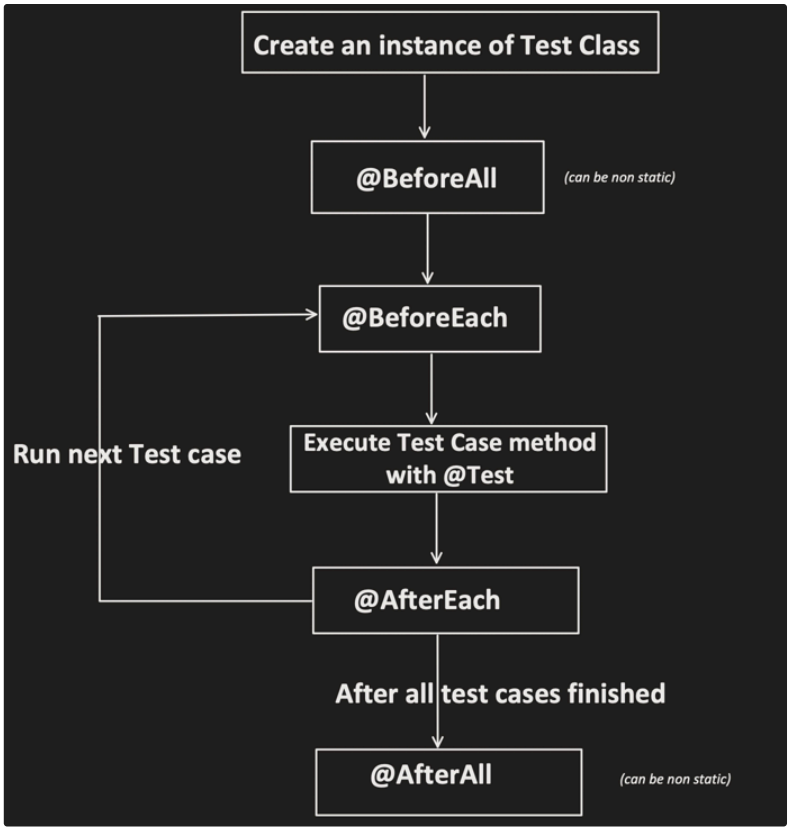
- Test Class instance is created for each Test Method.



Lifecycle.PER_CLASS

- Test Class instance is created only once.

Concept && Coding



@BeforeAll: runs once before all tests, usually for expensive setup that can be shared across all test cases like DB connection etc.

@BeforeEach: Runs before each test case, usually for setting fresh test data like creating new objects, adding some temp data in DB for testing etc.

@Test: actual test method

@AfterEach: Runs after each test case, to clean up the resource like close file, deleting temp data etc.

@AfterAll: runs once after all tests, clean up resources like closing DB connection etc.

pom.xml

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>

```

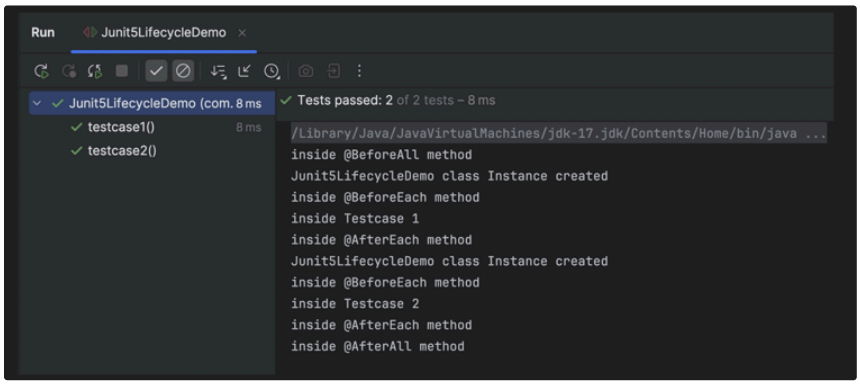
It transitively pulls in *"org.junit.jupiter:junit-jupiter"* and *org.junit.platform:junit-platform-launcher* dependency

Concept && Coding

Lifecycle.PER_METHOD

```java
public class Junit5LifecycleDemo {

    public Junit5LifecycleDemo() {
        System.out.println("Junit5LifecycleDemo class Instance created");
    }

    @BeforeAll
    static void beforeAll() {
        System.out.println("inside @BeforeAll method");
    }

    @BeforeEach
    void beforeEach() {
        System.out.println("inside @BeforeEach method");
    }

    @Test
    void testcase1() {
        System.out.println("inside Testcase 1");
    }

    @Test
    void testcase2() {
        System.out.println("inside Testcase 2");
    }

    @AfterEach
    void afterEach() {
        System.out.println("inside @AfterEach method");
    }

    @AfterAll
    static void afterAll() {
        System.out.println("inside @AfterAll method");
    }
}
```

Output:



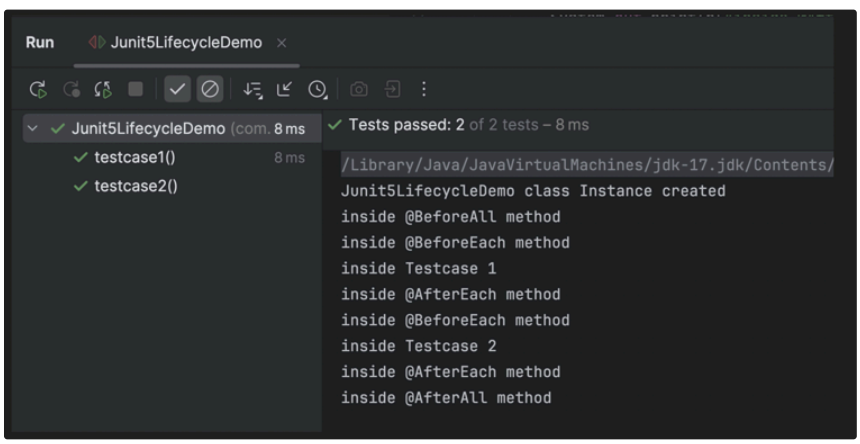
Lifecycle.PER_CLASS

```java
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class Junit5LifecycleDemo {

    public Junit5LifecycleDemo() {
```

Concept && Coding

```java
        System.out.println("Junit5LifecycleDemo class Instance created");
    }

    @BeforeAll
    void beforeAll() {
        System.out.println("inside @BeforeAll method");
    }

    @BeforeEach
    void beforeEach() {
        System.out.println("inside @BeforeEach method");
    }

    @Test
    void testcase1() {
        System.out.println("inside Testcase 1");
    }

    @Test
    void testcase2() {
        System.out.println("inside Testcase 2");
    }

    @AfterEach
    void afterEach() {
        System.out.println("inside @AfterEach method");
    }

    @AfterAll
    void afterAll() {
        System.out.println("inside @AfterAll method");
    }
}

```

Output:



Why Lifecycle.PER_METHOD is preferred, because:

- It provides test isolation. So they can run in any order.
- Easy debugging when any test case fails.
- Safe for parallel execution.

Go for Lifecycle.PER_CLASS only when:

- You want to shared state across test cases.
- Your test code is thread safe.

Concept && Coding

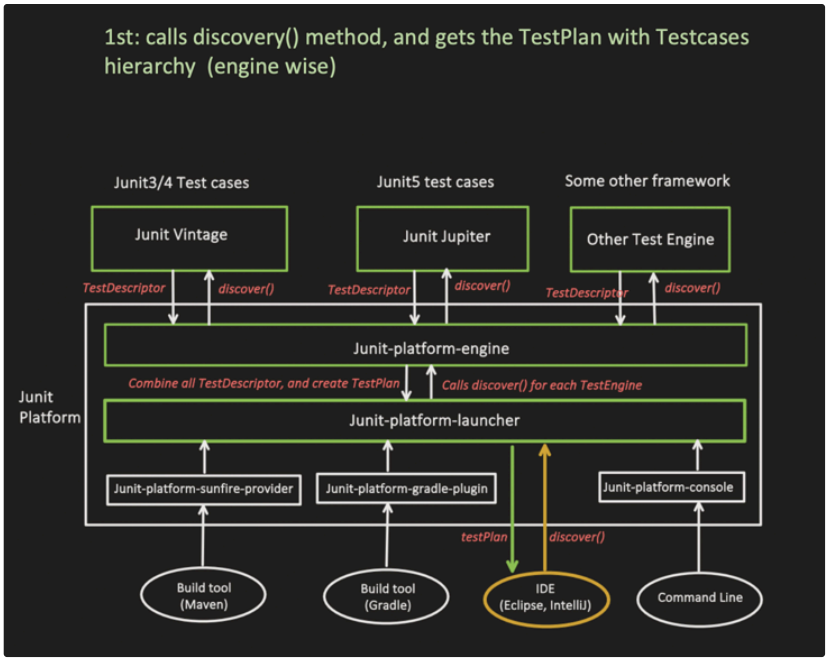
Multiple @BeforeAll / @BeforeEach / @AfterEach / @AfterAll can be used But:

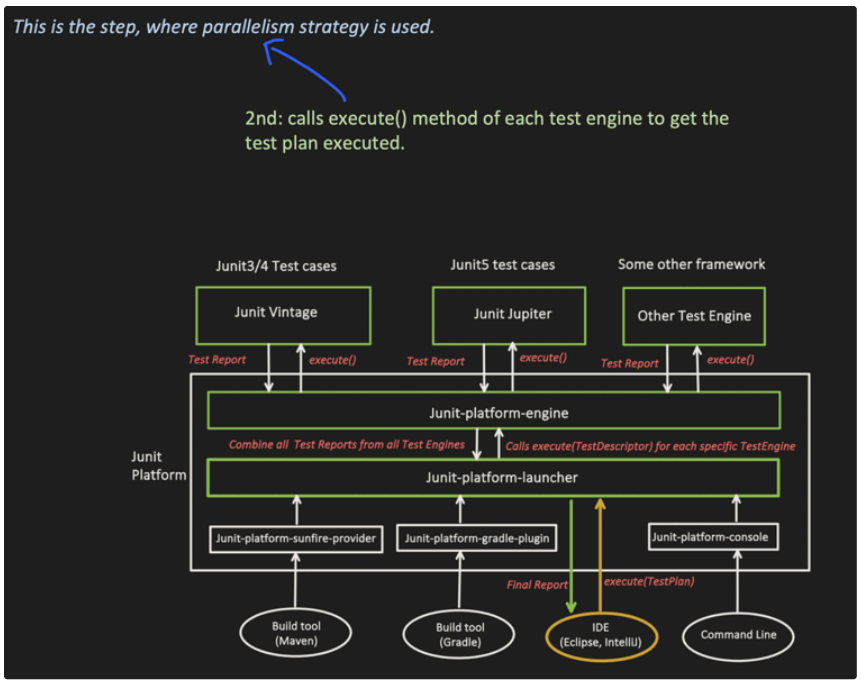
Order is not guaranteed between execution of these 2 methods.

By default, test cases runs sequentially, but we can also set our test cases to run in parallel:

From previous topic : Junit5 Architecture, its clear that, who helps test cases to run in parallel, its "PLATFORM LAUNCHER"

Concept && Coding






```java
public class Junit5LifecycleDemo {

    public Junit5LifecycleDemo() {
        System.out.println("Junit5LifecycleDemo class Instance created");
    }

    @BeforeAll
    static void beforeAll() {
        System.out.println("inside @BeforeAll method");
    }

    @BeforeEach
    void beforeEach() {
        System.out.println("inside @BeforeEach method");
    }

    @Test
    void testcaseA() throws InterruptedException {
```

Concept && Coding

```java
        System.out.println("inside testcaseA, executed by: " + Thread.currentThread().getName());
        Thread.sleep(1000);
        System.out.println("Finished testcaseA");
    }

    @Test
    void testcaseB() throws InterruptedException {
        System.out.println("inside testcaseB, executed by: " + Thread.currentThread().getName());
        Thread.sleep(1000);
        System.out.println("Finished testcaseB");
    }

    @AfterEach
    void afterEach() {
        System.out.println("inside @AfterEach method");
    }

    @AfterAll
    static void afterAll() {
        System.out.println("inside @AfterAll method");
    }
}

```

junit-platform.properties

```properties
#below configurations, junit platform launcher will apply to junit.jupiter (i.e. Jupiter Test engine only)

#Enables or disables parallel execution, default value is "false"
junit.jupiter.execution.parallel.enabled = true

#Run methods within a test class in parallel, default value is "same_thread" means sequentially
junit.jupiter.execution.parallel.mode.default = concurrent

#Multiple Test Classes can be executed in parallel, default value is "same_thread" means sequentially
junit.jupiter.execution.parallel.mode.classes.default = concurrent

#thread pool managing strategy:
#   "dynamic" (default) : thread pool size is computed dynamically based on system resource
#   "fixed" : we define the specific number of threads within a pool
junit.jupiter.execution.parallel.config.strategy = fixed

#used when "fixed" thread pool strategy is used, below it creates thread pool with 5 threads.
junit.jupiter.execution.parallel.config.fixed.parallelism = 5

```

Output:

Concept && Coding

```
Run    Junit5LifecycleDemo
Junit5LifecycleDemo 2 sec 38 ms      Tests passed: 2 of 2 tests – 2 sec 38 ms
    testcaseA()    1 sec 19 ms      /Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/
    testcaseB()    1 sec 19 ms      inside @BeforeAll method
                                    Junit5LifecycleDemo class Instance created
                                    Junit5LifecycleDemo class Instance created
                                    inside @BeforeEach method
                                    inside @BeforeEach method
                                    inside testcaseA, executed by:ForkJoinPool-1-worker-2
                                    inside testcaseB, executed by:ForkJoinPool-1-worker-1
                                    Finished testcaseA
                                    Finished testcaseB
                                    inside @AfterEach method
                                    inside @AfterEach method
                                    inside @AfterAll method
```

Follow AAA (Arrange-Act-Assert) Pattern for each Test Case:

```
@Test
void testUserWithAgeGreaterThan18() {

    // Arrange
    User user = new User( name: "TestUser",  age: 20);

    // Act
    boolean result = user.isAdult();

    // Assert
    assertTrue(result);
}
```

Setting up of object, mock data etc.

Invocation of actual method, which need to be tested

Check if the result matches the expectation.

Concept && Coding